

# 85. La caméra : c'est le monde qui bouge

Le héros ne se déplace pas — comme dans Mario.

## 1. Ce qu'il faut comprendre

Dans un jeu qui défile, le personnage **ne bouge pas à l'écran**. Il reste au milieu, et c'est le décor qui passe derrière lui. Mario n'avance pas : il piétine au centre pendant qu'un monde de plusieurs écrans glisse vers la gauche.

Il y a donc **deux espaces** : l'écran, qui fait 160 pixels de large, et le monde, qui fait ce qu'on veut.

`defiler(camera, 0)` dit simplement **quel morceau du monde on regarde**.

Et voici pourquoi cela tient en si peu de lignes : **les lutins ne défilent pas**. `defiler()` ne déplace que le fond ; un `sprite()` est posé sur l'écran, à la place où on l'écrit. Le héros au milieu, c'est une ligne, toujours la même :

```
sprite(0, 76, 112, HEROS);
```

Une seule variable suffit pour les deux rôles. `camera` est la fenêtre **et** la position du héros dans le monde : il est en `camera + 76`. Le jour où un ennemi arrive, lui vit dans le monde, et c'est à lui de se convertir — `sprite(1, xEnnemi - camera, 112, ENNEMI)`.

La preuve est à l'écran, et elle est gratuite : **même la consigne s'en va**. Elle est écrite dans le décor, donc elle voyage avec lui. Le bonhomme, lui, ne bouge pas d'un pixel.

### Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <defiler>` → [0.90.3. La fonction defiler\(\) — faire glisser le fond](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **845 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      57 o ← la console, à l'allumage — la mise en route de la console
main()         180 o ← démarrage — ta fonction
VBlank         8 o  ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte — attend de pouvoir écrire une case
AdresseCase    10 o ← poser() — trouve l'adresse d'une case de l'écran
EcrireTexte    10 o ← texte() — écrit un texte, lettre par lettre
RangerLutins   11 o ← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert 15 o ← démarrage — installe la copie des lutins en mémoire rapide
```

```

CopierTuiles      67 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte      14 o ← démarrage – vide les deux cartes de l'écran
LireManette       37 o ← bouton() – lit les huit boutons
Tuiles           320 o ← CopierTuiles – 17 lettres sur 43, et 3 tuiles dessinées
DonneesTransfert  10 o ← InstallerTransfert – la petite routine qui copie les lutins
"IL NE BOUGE PAS"  15 o ← texte() – un texte gravé
"CEST LE MONDE Q..." 17 o ← texte() – un texte gravé
"PASSE DERRIERE ..." 18 o ← texte() – un texte gravé

```

Lettres gravées : ABCDEGILMNOPQRSTU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

```

poser()   × 6   57 o sur place, dans main() + AdresseCase (10 o)
texte()   × 3   33 o sur place, dans main() + EcrireTexte (10 o)
sprite()  × 1   23 o sur place, dans main()
bouton()  × 2   10 o sur place, dans main() + LireManette (37 o)
defiler() × 1    7 o sur place, dans main()
image()   × 1    6 o sur place, dans main() + AttendreImage (28 o)

```

## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

#include <Tuile>      // un dessin de 8 × 8 pixels
#include <texte>      // écrit un texte à l'écran
#include <poser>      // pose une tuile sur une case du fond
#include <bouton>     // lit un bouton de la manette
#include <defiler>    // fait glisser tout le fond
#include <sprite>     // place un lutin de 8 × 8 au pixel près

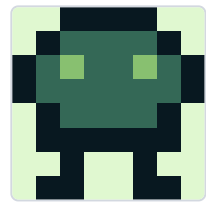
```

## 2. UN DESSIN — HEROS

```

Tuile HEROS = {
  "00333300",
  "03222230",
  "32122123",
  "32222223",
  "03222230",
  "03333330",
  "00300300",
  "03300330",
};

```



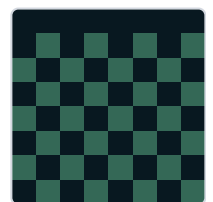
à l'écran

## 3. UN DESSIN — SOL

```

Tuile SOL = {
  "33333333",
  "32323232",
  "23232323",
  "32323232",
  "23232323",
  "32323232",
  "23232323",
  "32323232",
};

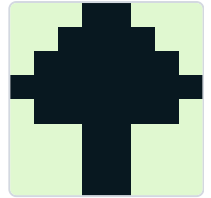
```



à l'écran

#### 4. UN DESSIN — ARBRE

```
Tuile ARBRE = {  
    "00033000",  
    "00333300",  
    "03333330",  
    "33333333",  
    "03333330",  
    "00033000",  
    "00033000",  
    "00033000",  
};
```



à l'écran

#### 5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

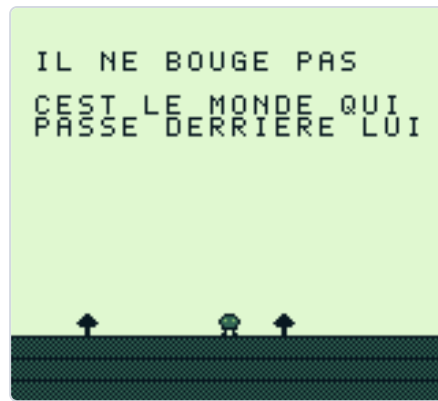
```
int main() {  
    uint8_t camera = 0;  
  
    texte(1, 2, "IL NE BOUGE PAS");  
    texte(1, 4, "CEST LE MONDE QUI");  
    texte(1, 5, "PASSE DERRIERE LUI");  
  
    for (uint8_t c = 0; c < 32; c++) {  
        poser(c, 15, SOL);  
        poser(c, 16, SOL);  
        poser(c, 17, SOL);  
    }  
  
    poser(3, 14, ARBRE);  
    poser(12, 14, ARBRE);  
    poser(25, 14, ARBRE);  
  
    while (true) {  
        image();  
  
        if (bouton(DROITE)) camera = camera + 1;  
        if (bouton(GAUCHE)) camera = camera - 1;  
  
        defiler(camera, 0);  
        sprite(0, 76, 112, HEROS);  
    }  
  
    return 0;  
}
```

### 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



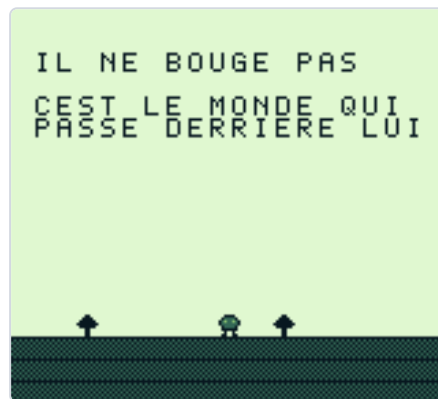
un tiers de seconde plus tard : le premier dessin est posé — image 20

## 4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé

## 5. Ce qu'on doit voir

Le bonhomme reste planté au milieu ; les arbres, le sol et même la consigne passent derrière lui.

## 6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le héros est posé au milieu de l'écran (76, 112)
- ✓ le monde est plus large que l'écran
- ✓ des arbres le jalonnent
- ✓ DROITE fait glisser le monde (0 vers 40)
- ✓ GAUCHE le ramène (40 vers 20)
- ✓ et le héros n'a pas bougé d'un pixel (76, 112)